# SaaS Complaint Management System

## نطاق المشروع

### ١. التعريف بالمصطلحات (Glossary/ Terminology ):

يحدد النظام الأدوار والمستويات التالية :

١. المنصة ( Platform) : النظام الرئيسي كخدمة (SaaS ) الذي يوفر بيئة متعددة المستأجرين .
٢. المستأجر ( Tenant) : الجهة \ الشركة \ المنشأة التي تشترك في المنصة وتستخدم خدماتها لإدارة شكاواها في بيئة معزولة .
٣. موظف المستأجر \ المتابع (Tenant Employee / Agent ) : الشخص التابع للشركة المستأجرة المسؤول عن إدخال الشكاوى الواردة ، متابعتها ، وتصنيفها .
٤. زبون المستأجر (Tenant`s Customer / End-User) : العميل النهائي للشركة المستأجرة (غير مرتبطة مباشرة بالمنصة في المرحلة الحالية ).

### ٢. الهدف من المشروع ( Project Objective) :

توفير منصة Multi-tenant SaaSمخصصة لإدارة الشكاوى ، تمكن الشركات (المستأجرين ) من الحصول على بيئات معزولة كلياً لإدارة ومتابعة الشكاوي الخاصة بها داخلياً عبر موظفيها، مع الاعتماد على الذكاء الاصطناعي (AI) لتصنف الشكاوي وتحديد الأقسام المعنية والأولوية .

### ٣. نطاق العمل الداخلي (In-Scope) :

١. إدارة المنصة والمستأجرين (Tenant Management & SaaS Core ):

- تخصيص البيئات : تسجيل المنشآت وإنشاء حسابات مخصصة لكل مستأجر ( Tenant Isolation) لضمان عزل تام للبيانات على مستوى قاعدة البيانات والواجهات .
- إدارة الخطط والاشتراكات : لوحة تحكم رئيسية (Super Admin Dashboard )لإدارة المستأجرين وتحديد الصلاحيات والخطط .
- إدارة الأقسام والموظفين : تمكين المستأجر من تعريف هيكلية الأقسام الداخلية الخاصة به والموظفين التابعين لها .

٢. إدارة الشكاوي ودورة حياتها (Complaint Management & Lifecycle) :

- إدخال الشكاوي : يدخل الموظف المستأجر (وليس زبون المستأجر في هذه المرحلة ) تفاصيل الشكوى الواردة إلى النظام وتحديد نطاقها وحالتها .
- تتبع وحل المشكلة : إمكانية متابعة الشكوى داخلية عبر الموظفين ،إضافة التعليقات، تغيير الحالات، وتسجيل الحل النهائي .
- إدارة الحقول النموذجية الديناميكية : تخصيص نموذج إدخال الشكوى بناءً على إعدادات الجهة المستأجرة .

٣. الذكاء الاصطناعي والتصنيف الآلي( AI-based Complaint Classification) :

- التصنيف الآلي للأقسام ( Department Classification) : استخدام نماذج الذكاء الاصطناعي مفتوحة المصدر (مثل Jina /Jeff Modelsأول النماذج المختصة ك LLM/SLM) لتحليل نص الشوى وتوجيهها تلقائياً إلى القسم المعني وفق الأقسام المعرفة لدى المستأجر .
- تصنيف الاستعجال \ الأولوية ( Urgency & Priority Classification) : تحليل درجة أهمية واستعجال الشكوى بناءً على المحتوى النصي.

٤. نطاق العمل الخارجي (Out-of-Scape) :

- بوابة الزبون المستأجر مباشرة ( Tenant Customer Portal) : إمكانية تقديم الشكوى وتتبعها مباشرة من قبل عميل المستأجر النهائي عبر واجهة خارجية .
- إدارة الأولوية المعقدة والقواعد التلقائية المتقدمة (Advanced Rules Engine) : الاكتفاء حالياً بالتصنيف الذكي للأقسام.
- استلام التقييم مباشرة من الزبون النهائي (Customer Feedback & Ratings).

٥. متطلبات البيانات والخدمة ( Database &System Requirements) :

١. Database Requirements:

- بناء المخطط بدعم كامل ل Multi-Tenancy مع Tenant ID Isolation.
- جداول مخصصة لإدارة ال Tenant، الأقسام ، الموظفين ، والشكاوي وحالاتها.

٢. Scope & Audit Requirements:

- سجل التدقيق (Audit Log) : تسجيل كل عملية إدخال أو تعديل على الشكاوى لحفظ تسلسل التتبع .
- دعم اللغات : واجهة عربية وانكليزية .
- إشعارات بريدية : إشعار الموظفين بالتغييرات أو التعيينات الجديدة .

## الدراسة التخطيطية للمشروع (Project Planning Study ) :

١. **إدارة التغييرات ( Change Management Plan)** : ضبط وتتبع أي تعديل طارئ أو مقترح على نطاق المشروع(Scope) ، المتطلبات، أو التصميم خلال فترة التنفيذ؛ لضمان عدم حدوث انحراف في الجدول الزمني أو الجدول المالي للمشروع.

- آلية طلب التغيير ( Change Request Process) مرور أي طلب تعديل بالخطوات الإلزامية الآتية:

١. التسجيل (Request): لا يتم اعتماد أي تعديل شفهياً .يجب تقديم طلب التغيير عبر وثيقة قالب مخصص (Change Request Form) أو تذكرة رسمية (Ticket) على نظام تتبع المهام (GitHub Issues / Trello)، تتضمن (اسم صاحب الطلب، تاريخ التقديم، سبب التغيير، والوصف التقني للتعديل).

٢. التحليل والأثر (Impact Analysis) : قبل الموافقة على أي تغيير، يقوم الفريق التقني بإجراء دراسة أثر تقنية تحدد:

- أثر الوقت والجدول الزمني :هل يتسبب التعديل بتأخير تسليم الـ Sprint الحالي؟
- التعقيد الفني وعزل البيانات :تقييم تأثير التعديل على أمان المستأجرين (Tenant Isolation)واستقرار النظام.
- استهلاك الموارد :تأثير التغيير على أداء خوادم الـ Backend وخدمات الذكاء الاصطناعي.

٣. الموافقة الرسمية ( Approval) : لا يتم الانتقال لتنفيذ أي تغيير في النطاق الأساسي (Scope) إلا بعد التوقيع والموافقة الخطية أو الرقمية من مشرف المشروع (الدكتور) وأعضاء الفريق المسؤولين عن التوثيق وإدارة المشروع .

- إدارة النسخ والتحديثات (Versioning Control) :

١. استراتيجية الفروع البرمجية (Branching Strategy) : الاعتماد على نظام ال Git مع هيكلية فروع واضحة ومنظمة .

٢. سجل التغييرات (Change Log) : توثيق كافة التعديلات البرمجية والهيكلية في جدول زمني خاص يوضح (رقم النسخة ، تاريخ التعديل ، وصف التغيير ، المستفيد منه).

٢. **إدارة الاختبار ( Test Management Plan)** : تحديد استراتيجيات ومستويات الاختبار لضمان جودة النظام ، امان البيانات ، دقة نماذج الذكاء الاصطناعي .

- أنواع ومستويات الاختبار ( Testing Levels) :

١. اختبار الوحدات (Unit Testing) : 

- الهدف :اختبار كل مكون برمجي بشكل مستقل (Isolated Components) للتأكد من خلوه من الأخطاء البرمجية قبل دمجه.
- الأداة والنطاق :استخدام مكتبات مثل PyTestلفحص واجهات الـFastAPI ، الخدمات المساعدة(Services) ، والتحقق من صحة مدخلات ومخرجات الدوال البرمجية (Functions).

٢. اختبار العزل والتكامل ( Integration & Multi-Tenancy Testing) :

- الهدف :التحقق القاطع من تطبيق مرشح معرف المستأجر (Tenant ID Filter) في كل استعلام قاعدة بيانات (PostgreSQL) .
- آلية الاختبار :محاكاة طلبات متزامنة من مستأجرين مختلفين للتأكد تحت أي ظرف وتحت ضغط البيانات من استحالة وصول مستأجر لبيانات مستأجر آخر، وضمان السرية التامة (Data Leakage Prevention).

٣. اختبار نموذج الذكاء الاصطناعي ( AI Model Evaluation) :

- الهدف :قياس كفاءة نموذج معالجة النصوص (LLM / Jina / Jeff أو مكتبات spaCy/NLTK ) في تصنيف الشكاوى وتحديد درجات الاستعجال.
- المعايير المعيارية المعتمدة(Evaluation Metrics) :
  - الدقة الكلية :(Accuracy) نسبة التصنيفات الصحيحة إجمالاً.
  - الدقة الجزئية :(Precision) دقة النموذج في نسب الشكاوى الموجهة لكل قسم صحيح.
  - الاسترجاع :(Recall) قدرة النموذج على رصد كافة الشكاوى التابعة لفئة معينة دون إغفالها.
  - مقياس :(F1-Score) التوازن الإحصائي بين Precision وRecall .
- آلية التحقق العملي :إدخال مجموعة اختبار قياسية (Test Dataset) مقدماً ومقارنة نتائج التصنيف التلقائي بالنتائج المتوقعة، مع قياس زمن استجابة الـ Microservice الخاص بالذكاء الاصطناعي.

٤. اختبار النظام الشامل واختبار قبول المستخدم (System & End-to-End Acceptance Testing):
الهدف : محاكاة دورة حياة الشكوى بالكامل(End-to-End Workflow) ؛ بدءاً من إدخال الشكوى عبر واجهة الموظف في React.js، مروراً بالتصنيف التلقائي الذكي وتوجيهها عبر لوحة Kanban Board، وحتى معالجتها وإغلاقها من قِبل القسم المختص وتوليد سجلات التدقيق (Audit Logs).

٣. **إدارة المشروع ( Project Management Plan)** :يتم فيها تحديد المنهجية ، الأدوات المستخدمة ، توزيع الأدوار بين أعضاء الفريق .

- منهجية التطبيق (Development Methodology) :
اتباع منهجية Agile \ Scrum المقسمة الى دورات عمل قصيرة ( Sprints مدتها من أسبوع الى أسبوعين لكل مرحلة ).
- أدوات إدارة المشروع والتواصل (Tools & Infrastructure) :

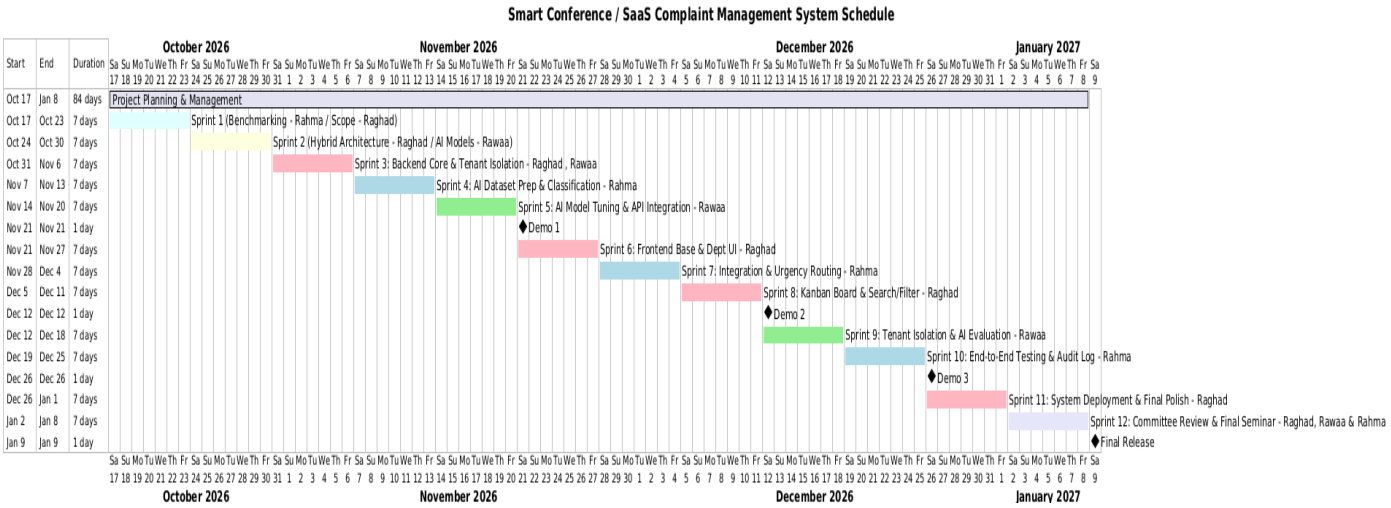
إدارة المهام ، إدارة الشفرة المصدرية والتكامل ، التوثيق والرسم المعماري .
Daily Standup : اجتماع قصير بين أعضاء الفريق (مسؤول الإدارة، مسؤول البرمجة، ومسؤول الذكاء الاصطناعي) لتحديد ما تم إنجازه، وما سيتم عمله، وعرض أي عقبات تقنية.
Sprint Planning &eview: لتحديد مهام كل أسبوع وعرض المخرجات أمام الفريق والمشرف (مثل عرض دراسة النماذج أو النماذج الأولية).

## ٤. خطة إدارة المخاطر (Risk Management):

- خطر عدم دقة نموذج الذكاء الاصطناعي في المراحل الأولى:
الحل : تجهيز بيانات تدريب كافية ومصنفة بدقة، وإتاحة خيار "التعديل اليدوي" للموظف في حال أخطأ النموذج في التصنيف .
- خطر تسريب البيانات بين المستأجرين(Tenant Data Leakage) :
الحل : اعتماد استراتيجيات صارمة في استعلامات قاعدة البيانات باستخدام مرشحات معرف المستأجر (Tenant ID Filter)واختبارها مبكراً .

## ٥. الخطة الزمنية (Project Timeline & Milestones) :

المخطط الزمني ل المنهجية المستخدمة وتقسيم المهام




## ٦. معمارية النظام (System Architecture ):

١. المعمارية المعتمدة : **المعمارية الهجينة ( Hybrid Layered-Microservice SaaS Architecture ):**
بناءً على مشروعنا ومتطلباته المعقدة التي تجمع بين تقديم المنصة كخدمة (**SaaS Multi-Tenant**) تتطلب عزل البيانات وأمان عالي ، وبين دمج تقنيات معالجة اللغة الطبيعية والذكاء الاصطناعي (**AI Text Categorization & Urgency Routing**)،تم اختيار معمارية هجينة (Hybrid Architecture ) تجمع بين قوتين أساسيتين:
- معمارية الطبقات المتعددة (Layered Architecture) : للنواة البرمجية الأساسية وإدارة المنصة والواجهات وقواعد البيانات .
- معمارية الخدمات المصغرة المستقلة (Microservices Architecture) : لخدمات الذكاء الاصطناعي ومعالجة النصوص.

٢. الهيكلية التفصيلية لطبقات النظام (Architectural Layers ) :
يتم تقسيم النظام إلى أربع طبقات ومكونات رئيسية لضمان فصل المسؤوليات (Separation of Concerns)

- أولاً : طبقة واجهة المستخدم (Frontend Layer – Presentation) :
تبنى باستخدام React.js لتوفير لوحات احكم تفاعلية وسريعة الاستجابة ، وتتضمن واجهة Kanban Board لإدارة الشكاوي ، ولوحات خاصة للمشرفين والموظفين والعملاء .
- ثانياً : طبقة الباك اند والنواة البرمجية (Core Backend Layer ) :
  - تبنى باستخدام Fast API/ Python وتتولى إدارة منطق العمل ( Business Logic ) ، المصادقة وإدارة الصلاحية (Authentication & Authorization) ، تطبيق سجلات التدقيق (Audit Logs) ، وإدارة دورة حياة الشكوى .
  - تتولى هذه الطبقة مهمة عزل بيانات المستأجرين (Tenant Isolation) عبر مرشحات صارمة (Tenant ID Filter ) في كافة الاستعلامات لضمان عدم تسريب البيانات بين الجهات المختلفة .
- ثالثاً : حدمة الذكاء الاصطناعي المصغرة (AI Microservice Layer ) :
  - خدمة مستقلة ومفصولة تم بناء ال API الخاص بها باستخدام Python /Fast API .
  - تتولى مهام معالجة النصوص ، واستدعاء وتدريب نماذج التصنيف (مثل Jeff / Jina / LLM) ،وارجاع تصنيف الشكوى ودرجة الاستعجال تلقائياً للنظام الأساسي مع إتاحة التعديل اليدوي للموظف عند الحاجة .
- رابعاً : طبقة التخزين وقواعد البيانات (Data Storage Layer ) :
اعتماد قاعدة البيانات متوافقة مع متطلبات ال Multi-Tenancy (مثل PostgreSQL) لتخزين بيانات الشكاوى والمستخدمين والمستأجرين بسجلات معزولة ومحمية .

٣. أسباب اختبار هذا المعمارية (Architectural Rationale ):

١. حماية وعزل بيانات المستأجرين (Tenant Isolation & Data Security ) :تسمح طبقة الباك اند المركزية بالتحكم الصارم بمرشحات قاعدة البيانات (Tenant ID Filter ) ومنع أي وصول غير مصرح به لبيانات مستأجر آخر تحت أي ظرف.

٢. استقلالية وكفاءة نماذج الذكاء الاصطناعي :(AI Performance & Isolation) نماذج الذكاء الاصطناعي ومعالجة النصوص تستهلك موارد عالية (CPU/GPU) وقد تتطلب وقتاً أطول لمعالجة الشكاوى .فصل الـ AI كـ Microserviceمستقل يضمن عدم تأثر استجابة وسرعة باقي صفحات ويب المنصة لو حدث ضغط على نموذج التصنيف.

٣. العمل بالتوازي والتطوير التراكمي :(Agile Parallel Development) تتيح هذه المعمارية الهجينة لفريق العمل البرمجي التطور على التوازي؛ حيث يعمل مسؤول البرمجة والـ Backend على النواة وقواعد البيانات والواجهات، بينما يعمل مسؤول الذكاء الاصطناعي على تدريب وتطوير النماذج وإعادة ضبطها دون تسبيب أي تعارض في الشفرة المصدرية(Codebase) .

٤. قابلة التوسع والتحسين المستقبلي :(Scalability & Maintainability) في حال الحاجة لتحديث نموذج الذكاء الاصطناعي أو استبداله بنموذج أحدث، يتم ذلك داخل الـ Microservice الخاصة به فقط دون الحاجة لتعديل أو إيقاف سيرفر الباك اند والواجهات الأساسية.

## ٧. لغات البرمجة والأدوات البرمجية المستخدمة(Tech Stack & Tools) :

١. لغات البرمجة(Programming Languages) : (Python للباك اند والذكاء الاصطناعي) لغة الأساس للمشروع بالكامل، حيث تم استخدامها لبناء النواة البرمجية ولخدمة الذكاء الاصطناعي لضمان التوافق التام وسهل دمج المكتبات.

٢. أطار العمل البرمجية(Frameworks) :

- Fast API:إطار العمل الأساسي لـ Backend والـAI Microservice ، ويتميز بسرعته العالية جداً ودعمه التلقائي لتوثيق الـAPIs (Swagger) .
- React.jsمع (Tailwind CSS): لبناء واجهات المستخدم (Frontend) التفاعلية، لوحات التحكم، وواجهة Kanban المخصصة لإدارة الشكاوى بمرونة وسرعة.

٣. قواعد البيانات وإدارة التخزين(Databases) : PostgreSQL : لتخزين بيانات الشكاوى، المستخدمين، والمستأجرين، مع دعم تطبيق استعلامات العزل الصارمة (Tenant ID Filter).
٤. أدوات الذكاء الاصطناعي ومعالجة النصوص(AI & NLP Tools) : مكتبات معالجة اللغة الطبيعية (NLP) مثل spaCy أو NLTKأو نماذج Hugging Face / Jina / Jeff لتنفيذ عملية تصنيف الشكاوى وتحديد درجات الاستعجال تلقائياً.
٥. أدوات إدارة المشروع، النسخ، والبيئة الهندسية(Tools & DevOps) :
- GitHub : لإدارة الشفرة البرمجية، تتبع النسخ، واتباع استراتيجية تفريغ واضحة (Development, Main, Feature Branches).
- GitHub Projects: لتتبع المهام، إدارة الـTickets ، وتوزيع الـ Sprints على أعضاء الفريق.
- Draw.io & PlantUML : لتوثيق ورسم المعمارية، مخططات الـUse Case ، وخط سير العمليات.(Workflows) .

**الفاعلون الأساسيون داخل نطاق النظام(In-Scope Actors)**

١. **المشرف الرئيسي للمنصة(Super Admin)**

- **الوصف :**المشرف المسؤول عن إدارة المنصة ككل على مستوى نظام الـ .SaaS
- **المهام والمسؤوليات:**
  - تسجيل وإدارة الشركات المستأجرة .(Tenants)
  - تحديد وتفعيل خطط الاشتراكات والصلاحيات لكل مستأجر .
  - مراقبة أداء المنصة وعزل البيئات .

٢. **مدير المستأجر / مشرف الشركة(Tenant Admin)**

- **الوصف :**الشخص المسؤول عن إدارة حساب الشركة أو المنشأة المستأجرة داخل بيئتها المعزولة .
- **المهام والمسؤوليات:**
  - تعريف وإدارة الهيكلية التنظيمية للأقسام الداخلية الخاصة بالشركة .
  - إدارة حسابات موظفي الشركة (إضافة، تعديل، إلغاء تفعيل) وتعيين أدوارهم .
  - ضبط وتخصيص الحقول النموذجية الديناميكية لنموذج إدخال الشكاوى .

٣. **موظف المستأجر / المتابع(Tenant Employee / Agent)**

- **الوصف :**الموظف التابع للشركة المستأجرة والمخول بالتعامل مع الشكاوى .
- **المهام والمسؤوليات:**
  - إدخال تفاصيل الشكاوى الواردة يدوياً إلى النظام وتحديد نطاقها وحالتها الأولية .
  - متابعة الشكاوى المسندة إليه أو لقسمه عبر لوحة **Kanban Board**.
  - إضافة التعليقات والملاحظات وتحديث حالة الشكوى وتسجيل الحل النهائي .
  - التعديل اليدوي لتصنيف القسم أو درجة الاستعجال إذا أخطأ نموذج الذكاء الاصطناعي .

٤. **خدمة الذكاء الاصطناعي / الفاعل البرمجي(AI Microservice System Actor)**

- **الوصف :**نظام فرعي ذكي يمثل فاعلاً برمجياً (System Actor) يعمل آلياً دون تدخل بشري مباشر .
- **المهام والمسؤوليات:**
  - استقبال نص الشكوى عند إدخالها ومعالجتها بلغة عربية عبر نموذج **AraBERT**
  - التصنيف الآلي للشكوى وتوجيهها للقسم المعني وفق أقسام المستأجر .
  - تحديد درجة الاستعجال والأولوية تلقائياً بناءً على محتوى النص

## المتطلبات الوظيفية :

### أولاً: إدارة المنصة والمستأجرين(Tenant Management & SaaS Core) :

١. **تسجيل المنشآت والمستأجرين** :يجب أن توفر المنصة آلية لتسجيل الشركات والمنشآت الجديدة وإنشاء بيئة معزولة كلياً لكل مستأجر (Tenant Isolation) على مستوى قواعد البيانات والواجهات .

٢. **عزل بيانات المستأجرين (Schema-per-Tenant)** :اعتماد نموذج العزل (Schema-per-Tenant) لتوفير عزل منطقي ضمن مساحة أسماء مستقلة (Namespace) لكل مستأجر، لمنع تداخل أو تسريب البيانات بين المستأجرين حتى في حال حدوث خطأ في الاستعلامات .

٣. **لوحة تحكم المشرف الرئيس (Super Admin Dashboard) :** توفير لوحة تحكم مركزية للمشرف الرئيسي لإدارة المستأجرين، مراقبة حالة الاشتراكات، وتحديد الصلاحيات والخطط لكل جهة .

٤. **إدارة الخطط والاشتراكات** :إمكانية تفعيل أو تجميد أو تعديل خطط الاشتراك الخاصة بكل مستأجر بناءً على الصلاحيات الممنوحة في النظام .

٥. **هيكلية الأقسام الداخلية** :تمكين كل مستأجر (شركة) من تعريف وإدارة هيكلية الأقسام الداخلية الخاصة به بشكل مستقل تماماً (مثل قسم الدعم الفني، المبيعات، الشؤون المالية، إلخ) .

٦. **إدارة موظفي المستأجر (Tenant Employees/Agents) :** توفير واجهة لإضافة، تعديل، أو حذف حسابات موظفي المستأجر المسؤولين عن متابعة وإدخال الشكاوى .

٧. **إدارة الأدوار والصلاحيات (RBAC)** : تحديد صلاحيات دقيقة لكل مستخدم داخل المنشأة (مثل صلاحية إدخال شكوى، تعديل حالة، أو إغلاقها).

### ثانياً: إدارة الشكاوى ودورة حياتها (Complaint Management & Lifecycle) :

٨. **إدخال الشكاوى اليدوي** :تمكين موظف المستأجر المعتمد من إدخال تفاصيل الشكوى الواردة يدوياً إلى النظام مع تحديد نطاقها وحالتها الأولية .

٩. **إدارة الحقول النموذجية الديناميكية** :السماح بتخصيص نموذج إدخال الشكوى ديناميكياً بناءً على إعدادات ومتطلبات كل جهة مستأجرة .

١٠. **تتبع دورة حياة الشكوى** :تتبع حالة الشكوى بدقة منذ لحظة إدخالها مروراً بمعالجتها وحتى إغلاقها النهائي داخل النظام .

١١. **لوحة تفاعلية لإدارة الشكاوى (Kanban Board):** توفير واجهة تفاعلية باستخدام لوحة Kanban لإدارة وتوزيع الشكاوى بين الموظفين والأقسام بمرونة عالية .

١٢. **إضافة التعليقات والملاحظات** :إمكانية إدراج تعليقات نصية ومتابعات داخلية ضمن تفاصيل الشكوى الواحدة من قبل الموظفين المعنيين .

١٣. **تغيير وتحديث حالات الشكاوي** :السماح بتحديث حالة الشكوى (مثل: قيد المعالجة، معلقة، تم الحل، مغلقة) مع حفظ التتبع الزمني .

١٤. **تسجيل الحل النهائي** :توفير حقل مخصص لتسجيل وصف الحل النهائي المعتمد للشكوى قبل إغلاقها رسمياً .

١٥. **البحث المتقدم والتصفية (Search & Filter)** : إمكانية البحث الفوري وتصفية الشكاوى بناءً على القسم، الأولوية، التاريخ، أو حالة الشكوى.

**ثالثاً: الذكاء الاصطناعي والتصنيف الآلي(AI-based Complaint Classification)**

١٦. **خدمة الذكاء الاصطناعي المستقلة (AI Microservice)** :فصل خدمة الذكاء الاصطناعي كـ Microserviceمستقل مبني باستخدام Python و FastAPI لضمان عدم تأثر أداء النظام الأساسي عند الضغط العالي .

١٧. **التصنيف الآلي للأقسام باستخدام AraBERT :** اعتماد نموذج AraBERT مضبوط-Fine) (tunedمع معالجة مسبقة محدودة (Minimal Preprocessing) لتحليل نص الشكوى وتوجيهها تلقائياً إلى القسم المعني بدقة عالية .

١٨. **طبقة التكييف التلقائي للأقسام (Adaptation Layer):** توفير طبقة برمجيّة تربط بين مخرجات نموذج الذكاء الاصطناعي وقائمة الأقسام الديناميكية الخاصة بكل مستأجر على حِدة .

١٩. **تصنيف درجة الاستعجال والأولوية (Urgency & Priority):** تحليل المحتوى النصي للشكوى تلقائياً لتحديد مستوى الأهمية والاستعجال (عالي، متوسط، منخفض) .

٢٠. **التعديل اليدوي للتصنيف** :إتاحة الخيار لموظف المستأجر لتعديل القسم أو الأولوية يدوياً في حال أخطأ نموذج الذكاء الاصطناعي في التصنيف التلقائي .

**رابعاً: قواعد البيانات والخدمات والتدقيق(Database & System Requirements) :**

٢١. **دعم تعدد المستأجرين في قاعدة البيانات** :بناء مخطط قاعدة البيانات (PostgreSQL) بدعم كامل لمتطلبات تعدد المستأجرين مع ربط كل جدول بـ .Tenant ID Isolation

٢٢. **سجل التدقيق الشامل (Audit Log):** تسجيل كل عملية إدخال، تعديل، أو حذف على الشكاوى لحفظ تسلسل التتبع ومعرفة المستخدم المسؤول عن الإجراء .

٢٣. **دعم اللغات المتعددة** :توفير واجهة مستخدم تدعم اللغتين العربية والإنكليزية بشكل كامل .

٢٤. **الإشعارات البريدية التلقائية** :إرسال إشعارات عبر البريد الإلكتروني للموظفين عند حدوث تغييرات هامة أو تعيين شكاوى جديدة لهم .

٢٥. **معمارية النظام الهجينة (Hybrid Architecture)** :ربط طبقة الواجهات (React.js) والنواة (FastAPI)مع خدمة الـ AI المصغرة بشكل تكاملي يضمن فصل المسؤوليات (Separation of Concerns).

**خامساً: إدارة الاختبارات والجودة(Testing Management)**

٢٦. **اختبار الوحدات البرمجية (Unit Testing):** إجراء اختبارات مستقلة للمكونات باستخدام مكتبة PyTestلفحص واجهات FastAPI والدوال البرمجية وضمان خلوها من الأخطاء .

٢٧. **اختبار العزل والتكامل (Integration & Multi-Tenancy Testing) :**محاكاة طلبات متزامنة للتأكد من تطبيق مرشح معرف المستأجر (Tenant ID Filter) بدقة واستحالة تسريب أي بيانات بين المستأجرين .(Data Leakage Prevention)

٢٨. **تقييم نموذج الذكاء الاصطناعي (AI Model Evaluation) :**قياس كفاءة نموذج التصنيف باستخدام معايير الدقة الكلية(Accuracy) ، الدقة الجزئية(Precision) ، الاسترجاع(Recall) ، ومقياس $(F1\text{-}Score)$عبر مجموعة اختبار قياسية .(Test Dataset)

٢٩. **اختبار النظام الشامل (End-to-End Acceptance Testing) :**اختبار دورة حياة الشكوى بالكامل؛ بدءاً من إدخالها عبر الواجهة، مروراً بالتصنيف التلقائي وإدارتها عبر لوحةKanban ، وصولاً إلى إغلاقها وتوليد سجلات التدقيق .

**سادساً: إدارة التغيير والنسخ(Change Management & Version Control) :**

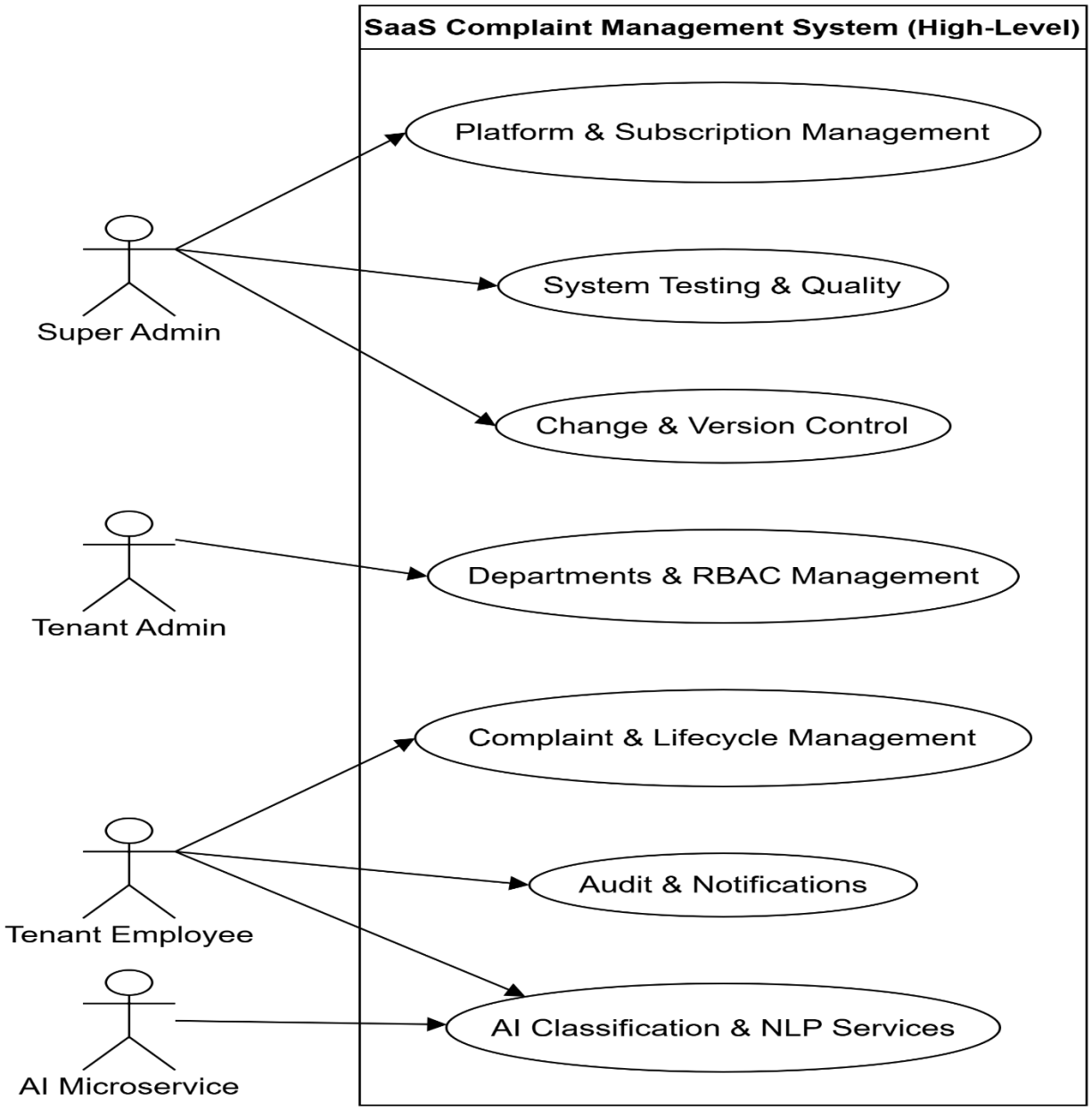
٣٠. **إدارة طلبات التغيير (Change Request Process):** توثيق أي تعديل طارئ على النطاق عبر قالب مخصص (Change Request Form) يتضمن سبب التغيير والوصف التقني، مع وجوب الموافقة الخطية أو الرقمية من المشرف والأعضاء .

٣١. **دراسة الأثر التقني للتغييرات** :تقييم أثر أي تعديل على الجدول زمني(Sprints) ، التعقيد الفني، عزل البيانات، واستهلاك موارد الخوادم قبل اعتماده .

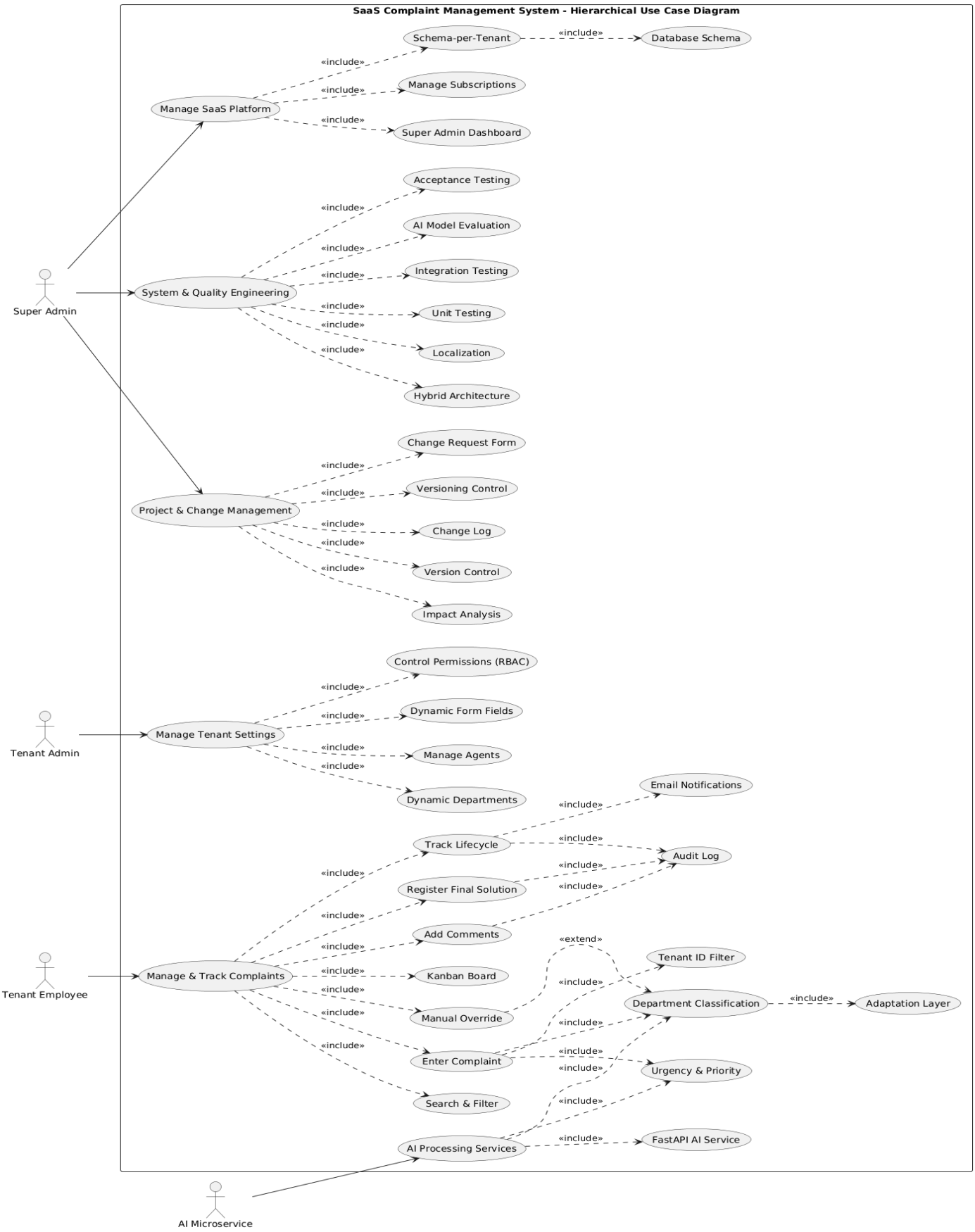
٣٢. **إدارة النسخ والفروع البرمجية (Branching Strategy):** اعتماد نظام Git مع هيكلية فروع منظمة (Development, Main, Feature Branches) لتتبع التعديلات البرمجية وتجنب التعارض .

٣٣. **سجل التغييرات البرمجية** :**(Change Log)** توثيق كافة التعديلات الهيكلية والبرمجية ضمن جدول زمني يوضح رقم النسخة، تاريخ التعديل، ووصف التغيير .

# High level Use Case Diagram :




# Low level Use Case Diagram :


SaaS Complaint Management System - Hierarchical Use Case Diagram
Super Admin
Tenant Admin
Tenant Employee
AI Microservice
Manage SaaS Platform
System & Quality Engineering
Project & Change Management
Manage Tenant Settings
Manage & Track Complaints
AI Processing Services
Schema-per-Tenant
Database Schema
Manage Subscriptions
Super Admin Dashboard
Acceptance Testing
AI Model Evaluation
Integration Testing
Unit Testing
Localization
Hybrid Architecture
Change Request Form
Versioning Control
Change Log
Version Control
Impact Analysis
Control Permissions (RBAC)
Dynamic Form Fields
Manage Agents
Dynamic Departments
Email Notifications
Track Lifecycle
Audit Log
Register Final Solution
Add Comments
Kanban Board
Manual Override
Tenant ID Filter
Department Classification
Adaptation Layer
Enter Complaint
Urgency & Priority
Search & Filter
FastAPI AI Service
«include»
«extend»

**المتطلبات غير الوظيفية :**

١. متطلبات الأمان وعزل البيانات(Security & Data Isolation Requirements)

- عزل بيانات المستأجرين التام :(Tenant Isolation) يجب أن يضمن النظام عزلاً كاملاً للبيانات على مستوى مساحة الأسماء باستخدام نمط(Schema-per-Tenant) ، بحيث يستحيل تداخل أو تسريب بيانات أي مستأجر لمستأجر آخر حتى في حال حدوث خطأ برمجي في الاستعلامات .
- منع تسريب البيانات :(Data Leakage Prevention) تطبيق مرشح معرف المستأجر الإلزامي (Tenant ID Filter)في كافة استعلامات قاعدة البيانات (PostgreSQL) لضمان السرية والخصوصية التامة .

٢. متطلبات الأداء وقابلية التوسع(Performance & Scalability Requirements)

- استقلالية أداء الذكاء الاصطناعي :(AI Performance & Isolation) فصل معالجة نماذج الذكاء الاصطناعي في خدمة مصغرة مستقلة (AI Microservice) لضمان عدم تأثر سرعة استجابة واجهات النظام وخوادم الـ Backend عند التعرض لضغط عالٍ في معالجة النصوص .
- التحكم بالكمون وزمن الاستجابة :(Latency Control) ضمان كفاءة زمن الاستجابة لنموذج التصنيف (AraBERT)لتوجيه الشكاوى بسرعة ودقة دون إبطاء عمل الموظفين .
- قابلية التوسع :(Scalability) تصميم بنية البيانات لتكون مرنة وقابلة للتطور المستقبلي (مثل الانتقال لاحقاً من عزل المخططات إلى قواعد بيانات مستقلة Database-per-Tenantللمستأجرين الكبار .(

٣. متطلبات المعمارية والموثوقية(Architectural & Reliability Requirements)

- المعمارية الهجينة :(Hybrid Architecture) الاعتماد على دمج معمارية الطبقات (Layered Architecture)للنواة والواجهات مع معمارية الخدمات المصغرة (Microservices) للذكاء الاصطناعي لتحقيق فصل تام للمسؤوليات .(Separation of Concerns)
- قابلية الصيانة والتطوير المتوازي :(Maintainability) تمكين الفريق التقني من تطوير وتحديث نماذج الذكاء الاصطناعي أو تعديلها ضمن الـ Microservice الخاصة بها بشكل منفصل دون الحاجة لإيقاف أو تعديل الشفرة المصدرية للباك إند الأساسي .

٤. متطلبات التدقيق والتتبع(Auditability & Traceability Requirements)

- سجل التدقيق الشامل :(Audit Log) الاحتفاظ بسجل رقمي دقيق ومستقل لكافة عمليات الإدخال، التعديل، وتغيير الحالات على الشكاوى لحفظ تسلسل التتبع ومعرفة المسؤول عن كل إجراء .
- إدارة التغييرات والنسخ :(Change & Version Control) توثيق كافة التعديلات البرمجية والهيكلية عبر سجل تغييرات زمني (Change Log) واتباع استراتيجية فروع منظمة في نظام Git لتفادي التعارض .

٥. متطلبات التوافقية وإمكانية الاستخدام(Localization & Usability Requirements)

- دعم اللغات :(Localization) يجب أن يدعم النظام واجهة مستخدم ثنائية اللغة (اللغتين العربية والإنكليزية) بشكل كامل .